

Scaling Networks with NAT and PAT

In most current network installations, Network Address Translation (NAT) is deployed along with private address space to allow the network administrator to efficiently and securely manage the corporate IP resources. This combination not only offers flexibility in the distribution of internal address space, but also contributes to the preservation of the current IPv4 address space.

Private Address Space (RFC 1918)

IETF RFC 1918 sets aside three blocks of IP addresses that are reserved for private use only and will not be assigned to any one individual or organization. Table 1-1 summarizes the RFC 1918 address space. Because these address blocks will remain unregistered, any organization is free to use them on its internal networks. Properly configured routers will not advertise this address space to the Internet, and packets that contain these addresses will be dropped as soon as they hit the Internet. The use of this address space on an organization’s internal network eliminates the waste and expense of maintaining large pools of routable addresses and allows the network administrator to design an IP addressing scheme that reflects the organization’s structure.

Table 1-1 RFC 1918 Address Space

Class	Range	CIDR Notation
A	10.0.0.0 to 10.255.255.255	10.0.0.0/8
B	172.16.0.0 to 172.31.255.255	172.16.0.0/12
C	192.168.0.0 to 192.168.255.255	192.168.0.0/16

Network Address Translation (NAT)

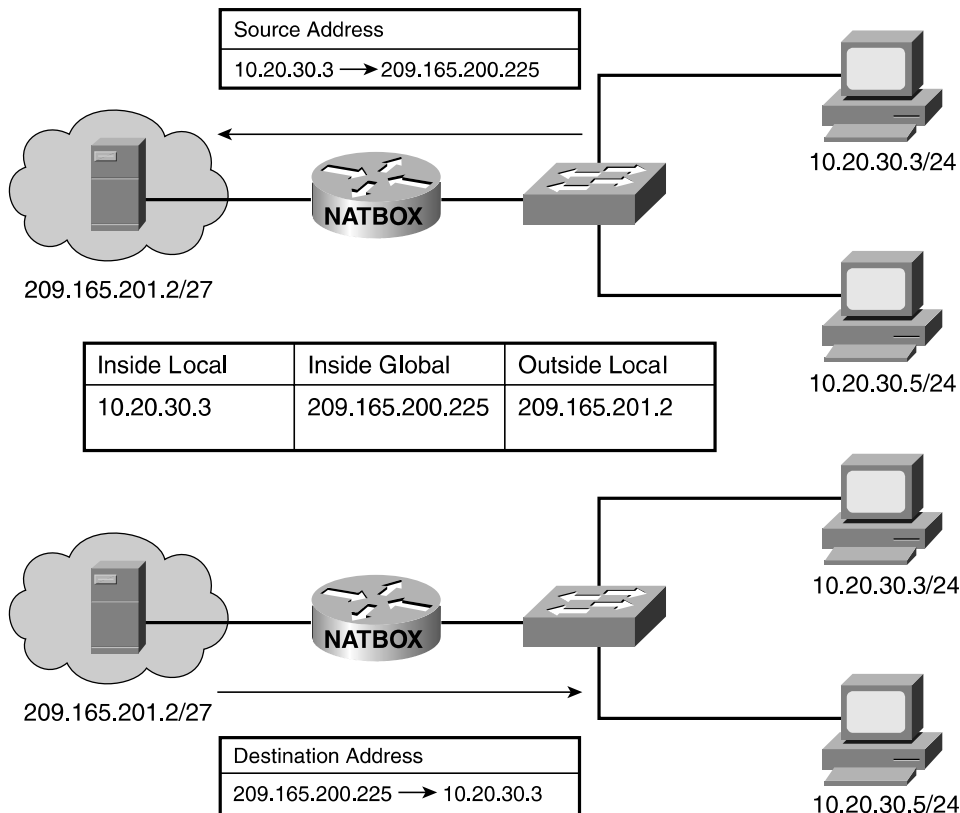
Deploying RFC 1918 addresses within an organization presents an interesting challenge. Because these addresses cannot be routed on the Internet, any packets that leave the internal network must have their source address replaced by one that is routable. Replies addressed to this routable address must then have the destination address translated back into the original private address before being placed on the internal network. This process of rewriting the Layer 3 information as a packet moves between the internal and external networks is known as *Network Address Translation (NAT)*. NAT, as discussed in IETF RFC 1631, allows an organization to present itself to the Internet with fewer IP addresses than there are nodes on the internal network. NAT usually maps an unregistered address used on an internal network to a registered address for use on the Internet, but it can also map between networks deploying routable or nonroutable addresses for certain applications. This chapter covers three forms of NAT:

- Static NAT
- Dynamic NAT
- Overloaded NAT or Port Address Translation (PAT)

NAT Terminology

Understanding the process of NAT and being able to troubleshoot it as issues arise requires a detailed understanding of the terminology used. NAT divides the network into two areas: inside and outside. The *inside network* is usually an organization's internal LAN, and the *outside network* is usually the Internet, although it could be any other network. *Local addresses* refer to a node on the network as seen by another node on the same network. *Global addresses* are how a node on one network is seen by a node on another network. For example, how a machine on the LAN appears to another machine on the same LAN is an *inside local address*, whereas that same machine would be seen by the *inside global address* from the outside world. A *translation* is the rewriting of information in the packet header from one IP address to another as the packet passes between the internal and external networks. The IOS maintains a *translation table* in RAM that tracks current translations. NAT simply changes the source IP address or port numbers on outbound traffic and the destination IP address or port number on inbound traffic. Figure 1-1 shows the inside local address being replaced with the inside global address on packets leaving the internal network. Then the inside global address is translated back to the inside local address in the replies before they are placed on the internal network.

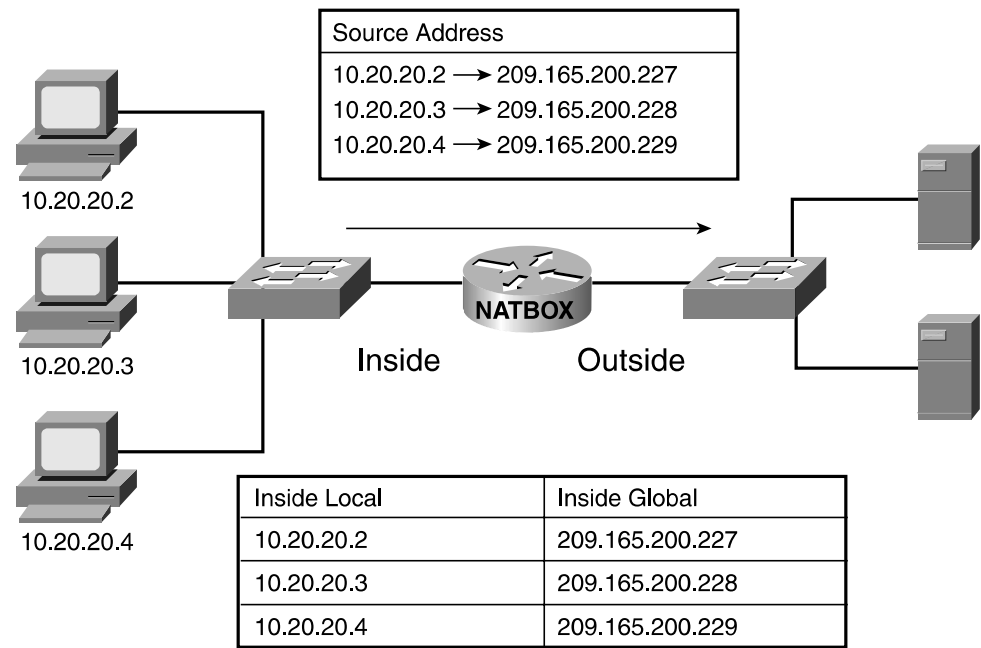
Figure 1-1 NAT Translation



Understanding and Configuring Static NAT

One form of NAT that is commonly implemented is known as *static NAT*. In this scenario, the network administrator manually configures a predefined one-to-one mapping of addresses between the internal and external networks. Static NAT has no conservation of IP address space, because each internal IP address must be mapped to a unique, routable external address. This mapping of addresses, illustrated in Figure 1-2, ensures that no packets are dropped due to lack of available address space. It also minimizes the delay introduced by building a dynamic translation.

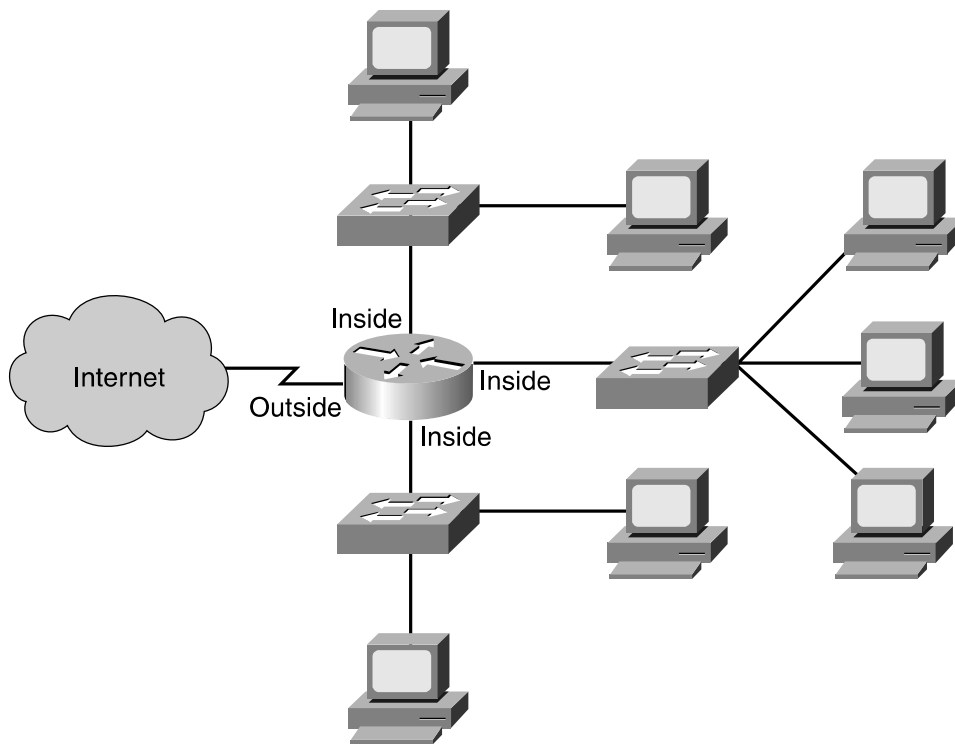
Figure 1-2 Static NAT



Static NAT is often deployed together with dynamic NAT, which is discussed in the next section. Static NAT is usually deployed to allow access to internal servers from the outside world. Servers that must be accessed from the outside are assigned an IP address consistent with that deployed on the internal network. Then a *static map* is created that maps a routable IP address to the internal address. This allows the server to be accessed using the inside local address from within the organization and also from outside the LAN using the inside global address. If the location of the internal server is changed, a new static map is created, making the change transparent to external users. In all forms of NAT, the hiding of internal address space from the external world provides a limited level of security in that no direct access to the internal network from the outside world is possible.

To configure static NAT, the inside and outside interfaces must be identified. Usually the Ethernet or LAN port is the *inside interface* and the serial or WAN port is the *outside interface*. This must be clearly stated, because all translations are done in the specified direction only. Figure 1-3 shows a typical assignment of inside and outside interfaces, and Example 1-1 provides the corresponding configuration. Inside and outside interfaces are defined using the **ip nat inside** and **ip nat outside** commands at the appropriate interface configuration prompt.

Figure 1-3 Multiple Inside Interfaces with NAT



Example 1-1 Configuring Inside and Outside Interfaces

```
interface FastEthernet0
  ip address 10.20.20.1 255.255.255.0
  ip nat inside
  speed auto
!
interface FastEthernet1
  ip address 10.20.30.1 255.255.255.0
  ip nat inside
  speed auto
!
interface FastEthernet2
  ip address 10.20.40.1 255.255.255.0
  ip nat inside
  speed auto
!
interface Serial0
  ip address 209.165.200.226 255.255.255.224
  ip nat outside
  no fair-queue
!
```

As soon as the inside and outside interfaces are established, the direction of translation and the actual translation are defined. The translation is an inside local address to an inside global address, as shown in the following syntax:

```
Router(config)#ip nat inside source static inside_local inside_global
```

For example, if you wanted to create three static translations to internal servers, as illustrated in Figure 1-2, you would issue the following commands:

```
Toronto(config)#ip nat inside source static 10.20.20.2 209.165.200.227
Toronto(config)#ip nat inside source static 10.20.20.3 209.165.200.228
Toronto(config)#ip nat inside source static 10.20.20.4 209.165.200.229
```

The NAT table in Example 1-2 clearly shows the mapping of the inside local address to the inside global address.

Example 1-2 Viewing the NAT Translation Table

```
Toronto#show ip nat translation
```

Pro	Inside global	Inside local	Outside local	Outside global
--	209.165.200.227	10.20.20.2	--	--
--	209.165.200.228	10.20.20.3	--	--
--	209.165.200.229	10.20.20.4	--	--

Depending on the deployment scenario, it may be desirable to NAT the inside source, outside source, or inside destination addresses. This may be accomplished by modifying the commands shown in Table 1-2.

Table 1-2 Selection of Address to NAT

Command	Description
ip nat inside source	Enables NAT of the inside source addresses
ip nat outside source	Enables NAT of the outside source addresses
ip nat inside destination	Enables NAT of the inside destination addresses



Lab 1-3 Configuring Static NAT Addresses (1.4.1c)

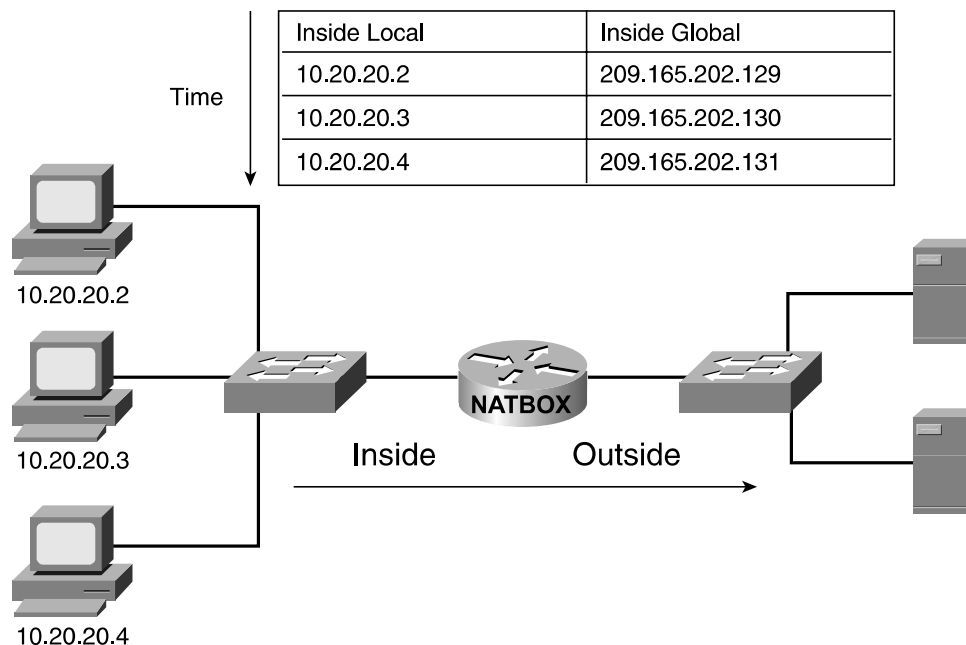
In this lab, you configure a router to use NAT to convert internal IP addresses, typically private addresses, into outside public addresses. You configure static mapping to allow outside access to an internal host.

Understanding and Configuring Dynamic NAT

Dynamic NAT eliminates the requirement for equal numbers of internal and external addresses by creating a pool of IP addresses that can be used for translation. Dynamic NAT does not allow for the assignment of a predefined address between the internal and external networks. Therefore, it is of little use for mapping internal servers to the outside world. This is why it is often deployed together with static NAT. Dynamic NAT does not guarantee that a translation will be successful if the pool of addresses is exhausted. Therefore, you must take care to provide an address pool of sufficient size to handle all possible simultaneous translations.

The most common use of dynamic NAT is to provide Internet connectivity to the employees in an organization that deploys RFC 1918 address space on the internal network. Because not all employees will require Internet access simultaneously, a pool of registered addresses smaller than the number of internal addresses can be configured. This reduces the expense of maintaining registered addresses for all employees, as would be required in a purely static NAT deployment.

Addresses are translated as packets leave the internal network. Translations are active only for the duration of the conversation or a configurable time period, whichever occurs first, as illustrated in Figure 1-4. After the reply is received, the translation is dissolved, and the address is returned to the NAT pool for reuse. The router maintains a record of active translations in the form of a translation table. Each *active translation* consumes approximately 160 bytes of DRAM. This translates to approximately 1.6 MB of DRAM for 10,000 active translations. Modern routers have more than enough memory to support thousands of NAT translations.

Figure 1-4 Dynamic NAT Translation

To create the pool of addresses used in dynamic NAT, use the following global configuration command:

```
ip nat pool name start-ip-address end-ip-address {netmask netmask | prefix-length prefix-length} {type rotary}
```

Giving the *NAT pool* a name that describes the location or task makes troubleshooting easier if multiple pools are to be configured.

In the example pictured in Figure 1-4, if you wanted to use the 209.165.202.129/27 subnet as a pool of addresses for dynamic NAT on the Toronto router, you would issue the following command:

```
Toronto(config)#ip nat pool border 209.165.202.129 209.165.202.158 netmask  
255.255.255.224
```

It is neither necessary nor advisable to translate all traffic found on the network, because this would waste valuable router resources. Traffic that should be translated is termed *interesting traffic* and is selected with the aid of an *access control list (ACL)*. NAT does not translate traffic not considered interesting as defined by the ACL.

If you wanted to allow only traffic originating on the 10.20.20.0/24 subnet of the Toronto network, you could use the following ACL to select the interesting traffic:

```
Toronto(config)#access-list 7 permit 10.20.20.0 0.0.0.255
```

The ACL is then tied to the NAT pool. The direction of translation is defined by the following global configuration command:

```
ip nat inside source {list {access-list-number | name} pool name [overload] |
static local-ip-address global-ip-address}
```

Continuing with our example, you would tie the ACL to the NAT pool using the following command:

```
Toronto(config)#ip nat inside source list 7 pool border
```

This command checks the source address on packets entering an interface that has been defined as an inside interface. If the source address matches the specified ACL, the traffic is identified as interesting, and the address is translated to an address found in the NAT pool before the traffic is moved out through the outside interface. If the source address does not match the ACL, the packet is not translated. A record of the translation is maintained in the NAT table. The router uses it to translate the IP address of replies before they are placed on the internal network.

Note

It is also possible to specify **outside** or the destination in the command to have complete control over the direction of translation. Although it's beyond the scope of this book, this material will be discussed in future CCNP-level courses.



Lab 1-1 Configuring NAT (1.1.4a)

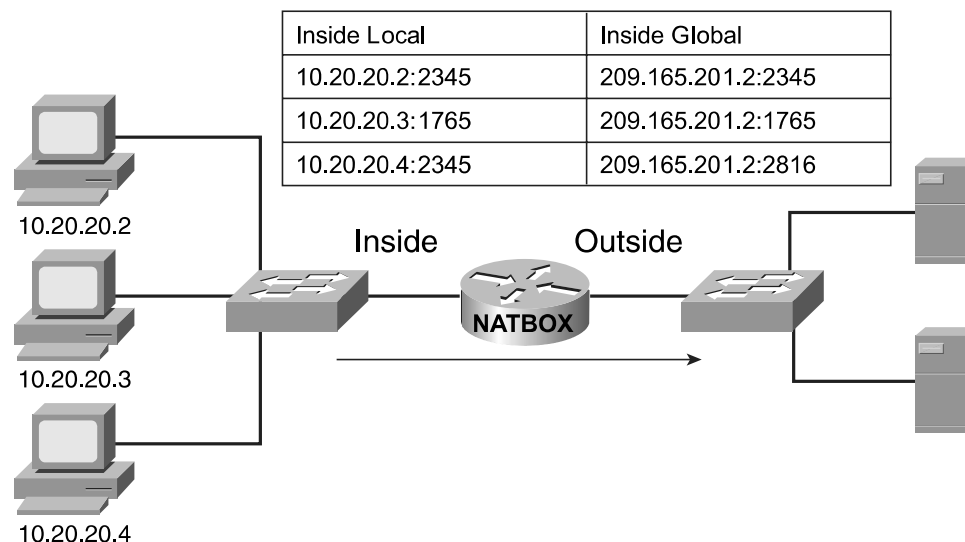
In this lab, you configure a router to use NAT to convert internal IP addresses into outside public addresses.

Understanding and Configuring Overloaded NAT

A modified form of dynamic NAT is *overloaded NAT*, which is also known as *Port Address Translation (PAT)*. With overloaded NAT, many addresses can be mapped to a single IP address using port numbers. This further reduces the number of addresses required by an organization to provide Internet connectivity to its users. Most large organizations now deploy PAT with a pool of IP addresses.

PAT divides the available ports per global IP address into three ranges of 0–511, 512–1023, and 1024–65535. When a translation is requested, PAT tries to preserve the *source port* on the global IP address. If the source port is unavailable, PAT begins searching from the beginning of the particular port range to find the first available port. If no ports are available in the selected range, the packet is dropped.

PAT can also be supplied with a pool of addresses to use. If PAT is requested and a pool of IP addresses has been configured, PAT first attempts to maintain the original source port on the first available global IP address. If this port is unavailable, PAT starts searching from the beginning of the relevant port range for an available port. If no ports are available in the relevant port range, PAT checks the second global IP address, trying to maintain the original source port. If the original port is unavailable, PAT again searches from the beginning of the relevant port range for an available port. This process is repeated until a new source port-IP address can be assigned or until PAT runs out of IP addresses to search. If PAT cannot find an available IP address-source port, it drops the packet. Figure 1-5 shows how PAT works.

Figure 1-5 Overloaded NAT or PAT

The configuration of PAT is nearly identical to the configuration of dynamic NAT, except that the keyword **overload** must be specified when setting up the translation. This keyword allows NAT to reassign ports on the source address while preserving the original IP address. It is also possible to overload an interface's IP address, commonly the WAN interface, instead of supplying a separate pool of addresses. Because the WAN port of most border routers is configured with an IP address that has been supplied by the ISP, using this address eliminates the requirement of obtaining a new IP address. The problem with this approach is that only one IP address is available, so if many translations must be made, one address may not suffice. To overload either a pool of addresses or the address assigned to an interface, use one of the following global commands:

```
ip nat inside source list acl_list_number pool pool_name overload
```

```
ip nat inside source list acl_list_number interface interface_name overload
```

Assume that you want to use the single address 209.165.201.2 to provide connectivity to the Internet. To accomplish this, you would need to define both the inside and outside interfaces, create an ACL to select interesting traffic, create a pool of addresses, and then apply the translation. It is possible to create an address pool that contains only one address by specifying the same address for both the beginning and ending address in the pool. Here are the final two steps:

```
Toronto(config)#ip nat pool border2 209.165.201.2 209.165.201.2 netmask
255.255.255.224
```

```
Toronto(config)#ip nat inside source list 7 pool border2 overload
```

If you decided to overload the IP address associated with the outside interface instead of creating a separate pool, the command would simply be as follows:

```
Toronto(config)#ip nat inside source list 7 interface serial 0 overload
```



Lab 1-2 Configuring PAT (1.1.4b)

In this lab, you configure a router to use PAT to convert internal IP addresses into outside public addresses.

Verifying NAT/PAT Functionality

A good place to start to verify NAT functionality or to troubleshoot a problematic implementation is with the **show ip nat statistics** command. This command provides information such as the number of packets that matched the ACL and were translated versus those that did not match the ACL, the location of the inside and outside interfaces, and the number of currently active translations. Quite often, this points to a problem with either the ACL or the assignment of interfaces. Example 1-3 provides sample output from the **show ip nat statistics** command.

Example 1-3 Viewing NAT Statistics

```
Toronto#show ip nat statistics
Total active translations: 0 (0 static, 0 dynamic; 0 extended)
Outside interfaces:
  Serial 0
Inside interfaces:
  FastEthernet0
Hits: 47 Misses: 3
Expired translations: 50
Dynamic mappings:
-- Inside Source
[Id: 1] access-list 7 pool toronto2 refcount 0
  pool toronto2: netmask 255.255.255.224
    start 209.165.201.2 end 209.165.201.2
    type generic, total addresses 1, allocated 0 (0%), misses 0
```

To view translations that are currently active, use the **show ip nat translations** command. This displays the contents of the translation table, allowing you to determine the correctness of the translations that are occurring. An improperly configured address pool could be handing out addresses outside of the desired range and impacting network accessibility. The addition of the keyword **verbose** to this command provides additional information such as how long the translation has been active. This can help diagnose issues related to network connectivity and improperly configured timeout values. Remember that NAT relies on an ACL to select traffic that is to be translated. By checking to see which packets matched the ACL, you can determine if any problems encountered are with the NAT translation process or merely the selection of interesting traffic by an improperly configured ACL. Example 1-4 provides sample output from the **show ip nat translations** command.

Example 1-4 The NAT Translation Table

```
Toronto#show ip nat translations
```

Pro	Inside global	Inside local	Outside local	Outside global
icmp	209.165.201.2:7972	10.20.20.1:7972	209.165.200.225:7972	209.165.200.225:7972
icmp	209.165.201.2:7973	10.20.20.1:7973	209.165.200.225:7973	209.165.200.225:7973
icmp	209.165.201.2:7974	10.20.20.1:7974	209.165.200.225:7974	209.165.200.225:7974
icmp	209.165.201.2:7975	10.20.20.1:7975	209.165.200.225:7975	209.165.200.225:7975
icmp	209.165.201.2:7976	10.20.20.1:7976	209.165.200.225:7976	209.165.200.225:7976

You can configure the length of time a translation remains active and stays in the translation table. Normally the translation is released when a reply is received, at which time the IP address is returned to the NAT pool. If a reply is not received, the entry stays in the table until it is removed by the *timeout value* or is manually removed by the network administrator. By default this timeout is set to 24 hours for TCP and 5 minutes for UDP. During the time that a translation remains in the NAT table, no other translation can use the IP address. If network delays or other issues arise that prevent the NAT process from receiving the reply, it is quite possible to run out of IP addresses that can be used for translations. This prevents users from accessing the Internet. Careful consideration of the timeout value is most important in dynamic NAT, which has a limited number of addresses available. In static NAT the mapping is predetermined, and PAT usually has large numbers of potential translations available.

You can configure a number of different timeout values in global mode to affect how NAT functions. Table 1-3 provides the commands necessary to set these timeout values along with the default values.

Table 1-3 Setting NAT Timeout Values

Command	Default Value
ip nat translation timeout <i>time</i>	24-hours
ip nat translation tcp-timeout <i>time</i>	24-hours
ip nat translation udp-timeout <i>time</i>	5-minutes
ip nat translation finrst-timeout <i>time</i>	60-seconds
ip nat translation dns-timeout <i>time</i>	60-seconds

To remove a translation that is currently active, you must clear it from the NAT table. The command used to remove an active translation from the NAT table depends on the type of translation that has occurred:

- **clear ip nat translation** * clears all dynamic NAT translations.
- **clear ip nat translation inside** *global_ip local_ip* {**outside** *local_ip global_ip*} removes a single dynamic translation entry.

- **clear ip nat translation** *protocol* **inside** *global_ip global_port local_ip local_port* {**outside** *local_ip local_port global_ip global_port*} removes the extended dynamic entry.

Caution

Exercise caution when clearing NAT translations on a production network. Clearing an active NAT translation forces the session to be restarted.

To verify that the correct translations are occurring, use the **debug ip nat [detailed]** command. The output from this command, as shown in Example 1-5, provides information detailing the translation of packets as they occur. In this output the source address is specified by an s, and d identifies the destination IP address. An arrow (->) indicates a translation that has occurred.

Example 1-5 Debugging NAT Translations

Toronto#**debug ip nat**

```
*Mar  1 14:01:01.620: NAT: s=10.20.20.1->209.165.201.2, d=209.165.202.130 [107]
*Mar  1 14:01:01.620: NAT*: s=209.165.202.130, d=209.165.201.2->10.20.20.1 [3926]
*Mar  1 14:01:01.624: NAT: s=10.20.20.1->209.165.201.2, d=209.165.202.130 [108]
*Mar  1 14:01:01.624: NAT*: s=209.165.202.130, d=209.165.201.2->10.20.20.1 [3927]
*Mar  1 14:01:01.628: NAT: s=10.20.20.1->209.165.201.2, d=209.165.202.130 [109]
*Mar  1 14:01:01.628: NAT*: s=209.165.202.130, d=209.165.201.2->10.20.20.1 [3928]
*Mar  1 14:01:01.628: NAT: s=10.20.20.1->209.165.201.2, d=209.165.202.130 [110]
*Mar  1 14:01:01.632: NAT*: s=209.165.202.130, d=209.165.201.2->10.20.20.1 [3929]
*Mar  1 14:01:01.632: NAT: s=10.20.20.1->209.165.201.2, d=209.165.202.130 [111]
*Mar  1 14:01:01.632: NAT*: s=209.165.202.130, d=209.165.201.2->10.20.20.1 [3930]
```

The first packet in any conversation must be *process-switched*, which establishes the translation entry. As soon as an entry exists in cache, any remaining packets in the conversation may take the *fast-switched* path. Packets that have taken the fast-switched path are flagged with an asterisk in the debug output.

NAT has contributed greatly to the conservation of existing address space by allowing the privatization of internal networks. It also allows an organization's internal network configuration to remain unchanged when moving from one service provider to another. Furthermore, it provides limited security by hiding the internal address space from the outside world.

Unfortunately, applications that rely on end-to-end IP addressing for their functionality do not work across a NAT router. If the remote IP address is contained in a packet's data portion instead of the header, NAT cannot occur. This is because NAT/PAT examines only the header. Any application that uses physical addresses instead of a qualified domain name will not reach destinations across a NAT router. This problem can sometimes be overcome using static mappings. The following applications do not always function across a NAT router, although methods exist to allow them to function:

- Routing table updates
- DNS zone transfers
- BOOTP
- Talk, ntalk

- Simple Network Management Protocol (SNMP)
- VoIP

**Lab 1-4 Verifying NAT and PAT Configuration (1.1.5)**

In this lab, you configure a router for NAT and PAT, test the configuration, and verify NAT/PAT statistics.

**Lab 1-5 Troubleshooting NAT and PAT (1.1.6)**

In this lab, you configure and troubleshoot a router for NAT and PAT.

Dynamic Host Configuration Protocol (DHCP)

Before the advent of *Dynamic Host Configuration Protocol (DHCP)*, all hosts on a network had to be manually configured with a separate, unique IP address before they could use network resources. Because of the static nature of the assignments, it was not possible to reuse any addresses, and any changes in network configuration required the network administrator to manually reconfigure each device. DHCP allows the automatic assignment of network configuration information to hosts on a network. With DHCP, addresses are assigned when required and are returned to a common pool when not in use. The DHCP protocol is described in RFC 2131 and is a successor to BOOTP.

BOOTP, as explained in RFC 951 and RFC 1542, was designed to enable diskless workstations to boot and request a minimal network configuration based on their MAC address. A BOOTP server is configured with a table of MAC addresses and the corresponding IP address that the administrator wants to assign. The server matches the Layer 2 address in the request to the Layer 3 address, and then this is sent back to the client in a reply. With BOOTP the same IP address is always handed to a workstation. This permanent assignment of IP addresses, static mappings, and a limit of four configuration parameters restricted BOOTP's usefulness and paved the way for a more versatile technique to dynamically configure network parameters.

DHCP reduces Internet access costs when NAT/PAT is not deployed by allowing address space to be dynamically assigned and reused. It also minimizes the time and expense of client configuration and provides a means for centralized management of IP information. There are many reasons why a network administrator may decide to run DHCP on a router rather than on a server-based system. Running DHCP on a router eliminates the need for extra hardware and allows DHCP to be run locally even if WAN connectivity is lost. On the downside, this decentralized method of running DHCP can complicate management and troubleshooting. Running a

DHCP server on a router forces a network administrator to use the Cisco IOS commands instead of a graphical configuration tool such as that provided by Windows. It also adds another service on a device that may at times already be overburdened.

DHCP provides a mechanism by which a client can be assigned an IP address for a limited amount of time known as the *lease* period. This allows a client to be assigned another address when it moves to a different subnet. It also lets the DHCP server assign the address to another client when the first client no longer requires it. This address reuse is a major factor in the conservation of IP addresses. If, at the end of the lease period, the client still requires the use of the same networking information, it may request a renewal of the lease from the DHCP server. DHCP additionally provides a method by which a client can gather other IP configuration information, such as the location of WINS and DNS servers, along with a domain name. The Cisco IOS implementation of DHCP supports more than 30 configuration parameters.

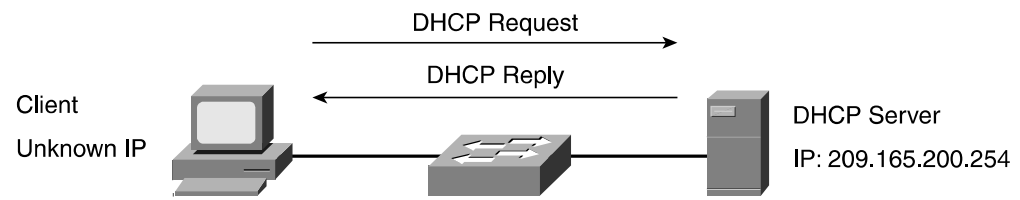
DHCP is not always possible or desirable. Servers on a network often require a static address, and some network administrators still prefer to statically assign network addresses to simplify management and security. In addition, the DHCP server must be available whenever an address is required, and end users must be prevented from manually configuring their network settings and potentially generating a conflict with dynamically assigned addresses.

DHCP supports three mechanisms for address allocation:

- **Automatic allocation** assigns a permanent address to the client.
- **Dynamic allocation** assigns an IP address for a fixed period of time or until the client relinquishes the address.
- With **manual allocation**, the administrator assigns an IP address to the client, and DHCP is used to convey the assigned address to the client.

DHCP, like its predecessor, BOOTP, uses UDP port 67 for requests and UDP port 68 for replies. The DHCP request is sent out as both a Layer 2 and Layer 3 broadcast for the destination but contains the client's MAC address as the source. The request asks the server to supply a client IP address and mask. The server responds to the request with a unicast message supplying the requested information. If the client is on the same segment as the DHCP server, the gateway IP address field is set to 0, and the IP address of the receiving interface is used to determine the pool of addresses that will be used. If the gateway IP address field is not 0, it is used to determine the subnet of the pool of addresses that should be distributed. In the reply, the destination MAC is the client's MAC address, and the destination IP address is one that has been supplied by the DHCP server. The Layer 2 and Layer 3 source address for the reply is that of the DHCP server. The client accepts this information and configures its network settings accordingly, as shown in Figure 1-6.

Figure 1-6 DHCP Requests and Replies



DHCP Request

MAC	IP	UDP	DHCP Request
SRC: Client DST: FF:FF:FF:FF:FF:FF	SRC: ? DST: 255.255.255.255	67	CIADDR: ? GIADDR: ? Mask: ? CHADDR: A

DHCP Reply

MAC	IP	UDP	DHCP Reply
SRC: Server DST: Client	SRC: 209.165.200.254 DST: 209.165.200.227	68	CIADDR: 209.165.200.227 GIADDR: ? Mask: 255.255.255.224 CHADDR: Client

CIADDR: Client IP Address, GIADDR: Gateway IP Address, CHADDR: Client Hardware Address

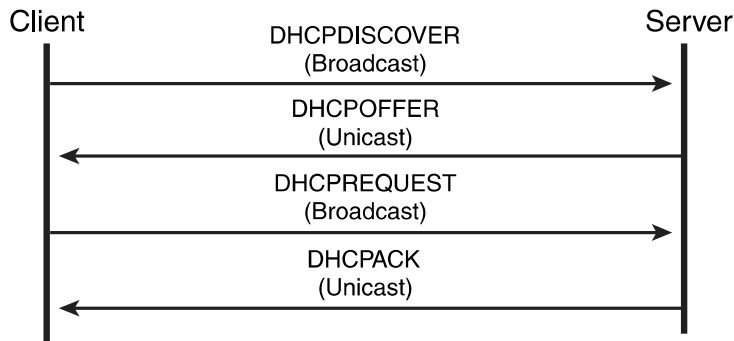
The DHCP Process

Properly troubleshooting the DHCP process requires a deeper understanding of how DHCP works. Figure 1-7 shows the complete process by which a DHCP configuration is requested:

1. The first step in the process is when the client machine issues a broadcast *DHCPDISCOVER* message to locate a DHCP server on the network.
2. Any DHCP servers that hear the message generate a unicast *DHCPOFFER* that contains configuration information such as an IP address, lease period, and domain name. If multiple DHCP servers are available on the network, the client may receive more than one *DHCPOFFER*. The client evaluates the offer and decides which one to accept. Usually the client accepts the first offer it receives. This is only an offer of configuration information at this point; the server has not assigned the information to the client.
3. The server usually reserves the information for the client until it has a chance to receive a *DHCPREQUEST* broadcast message from the client or it learns that the client has accepted configuration information from another server. The *DHCPREQUEST* is sent as a broadcast so that other DHCP servers on the network will not wait to hear from the client and will reclaim the addresses they had offered.

4. The server confirms that the offered information is still available and has been allocated to the client by issuing a **DHCPACK** unicast message to the client.

Figure 1-7 DHCP Process



If the **DHCPOFFER** is seen as invalid, the client refuses the offer by issuing a **DHCPDECLINE** broadcast message. If the client is slow in responding to the server, or an error occurs in the negotiation of parameters, the server issues a broadcast **DHCPNAK**. When the client no longer requires the IP address, it issues a **DHCPRELEASE** message telling the server that it may return the address to the pool for reassignment.

Configuring DHCP

By default the IOS DHCP service is enabled on routers. To enable the service if it has been disabled, use the global configuration command **service dhcp**. On larger networks, DHCP services are usually handled by one or more dedicated servers that provide a more robust solution than can be implemented on a single router. For security reasons, it's a good idea to disable all services on a network device except those required for network operability. Use the global command **no service dhcp** to disable the DHCP service on a router.

After the service is enabled on a router, you must create a pool or pools of addresses using the following command:

```
Router(config)#ip dhcp pool name
```

Troubleshooting is much easier if the name chosen for the **DHCP pool** is meaningful within the context of the network or organization.

After the address pool is defined, you must specify the addresses it contains. You can specify the network or subnet range available for the address pool by a network subnet address followed by either a valid subnet mask or the prefix length, as shown here:

```
Router(dhcp-config)#network network-number [mask | /prefix-length]
```

From this point, you can specify a number of optional configuration parameters, including the domain name, the location of the DNS and NetBIOS name servers, the default gateway (default-router), and the lease's duration. For the location of the DNS server, NetBIOS name